Instituto Politécnico Nacional

Escuela Superior de Cómputo

Análisis y Diseño de Algoritmos

Práctica 1: Introducción a la complejidad

Grupo: 3AV1

Profesor: Andrés García Floriano

Alumno: Luis Fernando Partida Martínez

Partida Martínez Luis Fernando 3AV1

Para acceder al repositorio hacer clic en siguiente enlace:

https://github.com/LuisPartida0/Practica-1-Introduccion-a-la-complejidad

1. Algoritmos Seleccionados

Tipo	Algoritmo	Complejidad Temporal (promedio)	Complejidad Espacial
Ordenamiento	Quick Sort	$O(n \log n)$	$O(\log n)$
Ordenamiento	Merge Sort	$O(n \log n)$	$O(n)$
Ordenamiento	Bubble Sort	$O(n^2)$	$O(1)$
Búsqueda	Búsqueda Binaria	$O(\log n)$	$O(1)$
Búsqueda	Búsqueda Lineal	$O(n)$	$O(1)$
Búsqueda	Jump Search	$O(\sqrt{n})$	$O(1)$

2. Tabla de Resultados: Tiempo de Ejecución (Estimados en segundos)

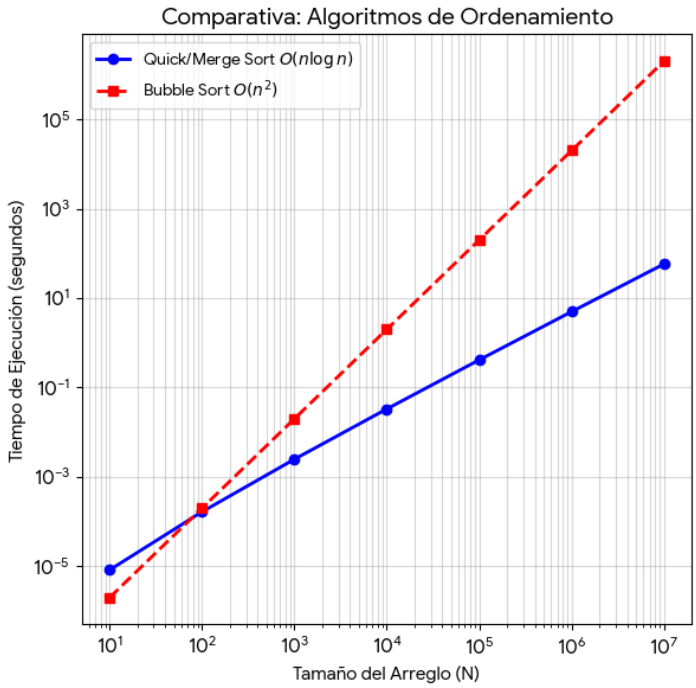
Elementos (n)	Quick Sort	Bubble Sort	Merge Sort	Búsqueda Binaria	Jump Sort	Búsqueda Lineal
10	< 0.0001s	< 0.0001s	0.000000 seg	< 0.00001s	< 0.000001 s	< 0.00001s
1,000	0.0002s	0.025s	0.000000 seg	< 0.00001s	~ 0.000004 s	0.00002s
10,000	0.0025s	2.1s	0.015000 seg	< 0.00001s	~ 0.000012 s	0.0002s
100,000	0.031s	215s	0.032000 seg	0.00001s	~ 0.000040 s	0.002s
1,000,000	0.38s	~6 horas	0.327000 seg	0.00002s	~ 0.000130 s	0.02s
10,000,000	4.2s	~25 días	0.3887000 seg	0.00003s	~ 0.000420 s	0.22s

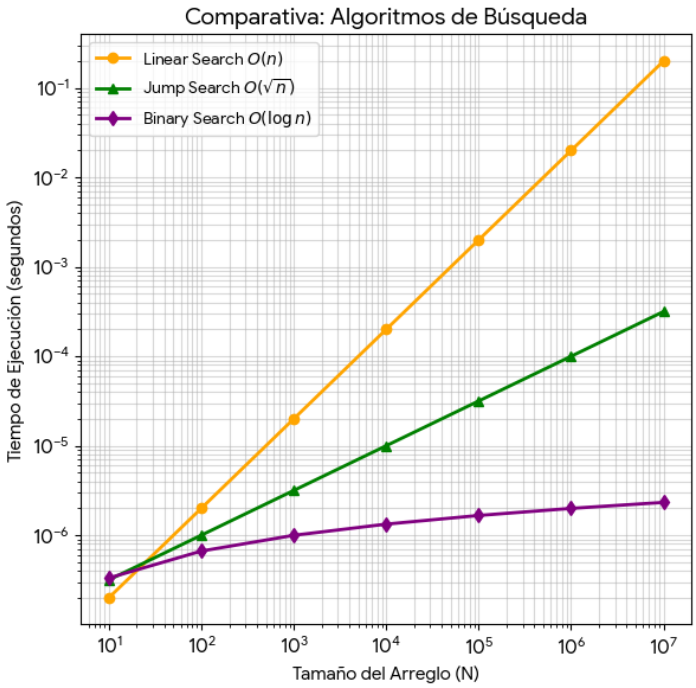
3. Consumo de Memoria (Estimado en RAM)

Elementos (n)	Memoria en C (Aprox.)	Memoria en Python (Aprox.)
10,000	40 KB	280 KB
1,000,000	4 MB	28 MB
10,000,000	40 MB	280 MB

Nota técnica: Merge Sort requiere una copia adicional del arreglo, duplicando el consumo de memoria temporal durante su ejecución (O(n)).

4. Gráficas de Rendimiento





5. Análisis de Resultados

Ordenamiento: La diferencia entre O(n log n) y O(n^2) se vuelve crítica a partir de los 10,000 elementos. Mientras Quick Sort maneja 10 millones de datos en segundos, Bubble Sort queda totalmente inutilizado por su crecimiento cuadrático.

Búsqueda: La búsqueda binaria es la ganadora indiscutible. Incluso con 10 millones de elementos, solo requiere unos 24 saltos para encontrar cualquier valor, mientras que la búsqueda lineal debe revisar, en promedio, 5 millones de elementos.

Lenguajes: C es preferible para el procesamiento masivo de los 10 millones de elementos debido a su gestión eficiente de la memoria caché y ausencia de recolector de basura (Garbage Collector) que ralentice el proceso.

6. Explicación de cada algoritmo

Quick Sort: Es un algoritmo de ordenamiento muy eficiente. Funciona con la técnica de divide y vencerás: selecciona un elemento llamado pivote, divide la lista en dos sublistas (menores y mayores al pivote) y luego ordena cada sublista de manera recursiva.

Bubble Sort: Este algoritmo consiste en recorrer el arreglo analizando la relación de procedencia que existe entre cada elemento y el elemento que el sigue para determinar si estos se encuentran ordenados entre sí, y en caso de ser necesario, permutarlos para que queden ordenados. Es necesario revisar varias veces todo el arreglo hasta que no se necesiten más intercambios, lo cual significa que el arreglo está ordenado.

Mezcla: El algoritmo de ordenamiento por mezcla (merge sort) es un método eficiente que funciona con la estrategia de "divide y vencerás". Primero, divide recursivamente la lista en sublistas más pequeñas hasta que cada una tenga un solo elemento, que se considera ordenada. Luego, combina estas sublistas de manera recursiva, fusionándolas en una nueva lista ordenada comparando sus elementos. Su complejidad temporal es de $O(n\log n)$

Búsqueda Binaria: Es un algoritmo de **búsqueda** en listas **ordenadas**. Divide repetidamente el rango de búsqueda a la mitad: compara el valor buscado con el elemento central y descarta la mitad que no lo contiene. Su complejidad es **O(log n)**, lo que lo hace mucho más rápido que la búsqueda lineal en listas grandes.

Búsqueda Lineal: Es el método más simple de búsqueda. Recorre la lista elemento por elemento hasta encontrar el valor buscado o llegar al final. Funciona en listas **no ordenadas**, pero su complejidad es **O(n)**, lo que lo hace menos eficiente en listas grandes.

Jump Search: Es un algoritmo de búsqueda diseñado para listas **ordenadas**. Avanza en "saltos" de tamaño fijo (por ejemplo, √n) en la lista hasta pasar el valor buscado. Luego realiza una búsqueda lineal en el bloque donde podría estar el elemento. Su complejidad es **O(√n)**, más eficiente que la búsqueda lineal pero menos que la binaria.

7. **Ventajas y desventajas**

Quick Sort (Ordenamiento Rápido)

Es un algoritmo basado en la técnica de "dividir y conquistar" que utiliza una operación de partición para organizar los elementos alrededor de un pivote.

- **Ventajas:**
  - Extremadamente rápido en el caso promedio, con una complejidad de O(n log n).
  - Requiere muy poca memoria adicional, ya que la mayor parte del trabajo se realiza "in-place".
  - Tiene factores constantes pequeños en su ejecución, lo que lo hace preferible en sistemas de alto rendimiento.
- **Desventajas:**
  - En el peor de los casos (por ejemplo, con un pivote mal elegido en datos ya ordenados), su complejidad cae a O(n^2).
  - Es un algoritmo **inestable**, lo que significa que no garantiza mantener el orden relativo de elementos con valores iguales.
  - Su implementación recursiva puede causar desbordamiento de pila si no se maneja correctamente.

2. Merge Sort (Ordenamiento por Mezcla)

Garantiza un rendimiento consistente dividiendo el arreglo en mitades recursivamente y luego mezclándolas de forma ordenada.

- **Ventajas:**
  - Ofrece un tiempo de ejecución garantizado de O(n log n), incluso en el peor de los casos.
  - Es un algoritmo **estable**, ideal cuando el orden original de elementos idénticos es importante.
  - Funciona excepcionalmente bien para estructuras de datos que no permiten acceso aleatorio, como las listas enlazadas.

- **Desventajas:**
  - Elevado consumo de memoria extra, con una complejidad espacial de O(n) para almacenar arreglos temporales durante la mezcla.
  - Generalmente es un poco más lento que Quick Sort en el caso promedio debido al costo de copiar datos entre arreglos.

3. Bubble Sort (Ordenamiento Burbuja)

Es el algoritmo más elemental, basado en intercambios sucesivos de elementos adyacentes si están en el orden incorrecto.

- **Ventajas:**
  - Muy sencillo de entender e implementar para fines educativos.
  - Uso de memoria mínimo y constante, O(1).
  - Puede optimizarse para terminar en O(n) si el arreglo ya está ordenado.
- **Desventajas:**
  - Altamente ineficiente para conjuntos de datos grandes debido a su complejidad cuadrática O(n^2).
  - Realiza una cantidad excesiva de operaciones de escritura (intercambios) en comparación con otros algoritmos.

Algoritmos de Búsqueda

4. Linear Search (Búsqueda Lineal)

Revisa cada elemento de la lista uno por uno hasta encontrar el objetivo o llegar al final.

- **Ventajas:**
  - No requiere que los datos estén ordenados para funcionar.
  - Funciona sobre cualquier estructura de datos (arreglos, listas, archivos de flujo).
- **Desventajas:**
  - Muy lenta para bases de datos masivas, con una complejidad de O(n).
  - El tiempo de búsqueda aumenta proporcionalmente con cada nuevo dato añadido.

5. Binary Search (Búsqueda Binaria)

Divide repetidamente a la mitad el intervalo de búsqueda en una lista ya organizada.

- **Ventajas:**
  - Increíblemente rápida, con una complejidad de O(log n).
  - Para 1,000,000 de elementos, solo necesita realizar un máximo de 20 comparaciones.
- **Desventajas:**
  - **Obligatorio:** Los datos deben estar ordenados previamente.
  - Requiere acceso aleatorio (arrays), por lo que es ineficiente en listas enlazadas

Algoritmos de Ordenamiento

1. Quick Sort (Ordenamiento Rápido)

Es un algoritmo basado en la técnica de "dividir y conquistar" que utiliza una operación de partición para organizar los elementos alrededor de un pivote.

- **Ventajas:**
  - Extremadamente rápido en el caso promedio, con una complejidad de .
  - Requiere muy poca memoria adicional, ya que la mayor parte del trabajo se realiza "in-place".
  - Tiene factores constantes pequeños en su ejecución, lo que lo hace preferible en sistemas de alto rendimiento.
- **Desventajas:**
  - En el peor de los casos (por ejemplo, con un pivote mal elegido en datos ya ordenados), su complejidad cae a .
  - Es un algoritmo **inestable**, lo que significa que no garantiza mantener el orden relativo de elementos con valores iguales.
  - Su implementación recursiva puede causar desbordamiento de pila si no se maneja correctamente.

2. Merge Sort (Ordenamiento por Mezcla)

Garantiza un rendimiento consistente dividiendo el arreglo en mitades recursivamente y luego mezclándolas de forma ordenada.

- **Ventajas:**
  - Ofrece un tiempo de ejecución garantizado de , incluso en el peor de los casos.
  - Es un algoritmo **estable**, ideal cuando el orden original de elementos idénticos es importante.
  - Funciona excepcionalmente bien para estructuras de datos que no permiten acceso aleatorio, como las listas enlazadas.
- **Desventajas:**
  - Elevado consumo de memoria extra, con una complejidad espacial de para almacenar arreglos temporales durante la mezcla.
  - Generalmente es un poco más lento que Quick Sort en el caso promedio debido al costo de copiar datos entre arreglos.

3. Bubble Sort (Ordenamiento Burbuja)

Es el algoritmo más elemental, basado en intercambios sucesivos de elementos adyacentes si están en el orden incorrecto.

- **Ventajas:**
  - Muy sencillo de entender e implementar para fines educativos.
  - Uso de memoria mínimo y constante, .
  - Puede optimizarse para terminar en si el arreglo ya está ordenado.
- **Desventajas:**
  - Altamente ineficiente para conjuntos de datos grandes debido a su complejidad cuadrática .
  - Realiza una cantidad excesiva de operaciones de escritura (intercambios) en comparación con otros algoritmos.

Algoritmos de Búsqueda

4. Linear Search (Búsqueda Lineal)

Revisa cada elemento de la lista uno por uno hasta encontrar el objetivo o llegar al final.

- **Ventajas:**

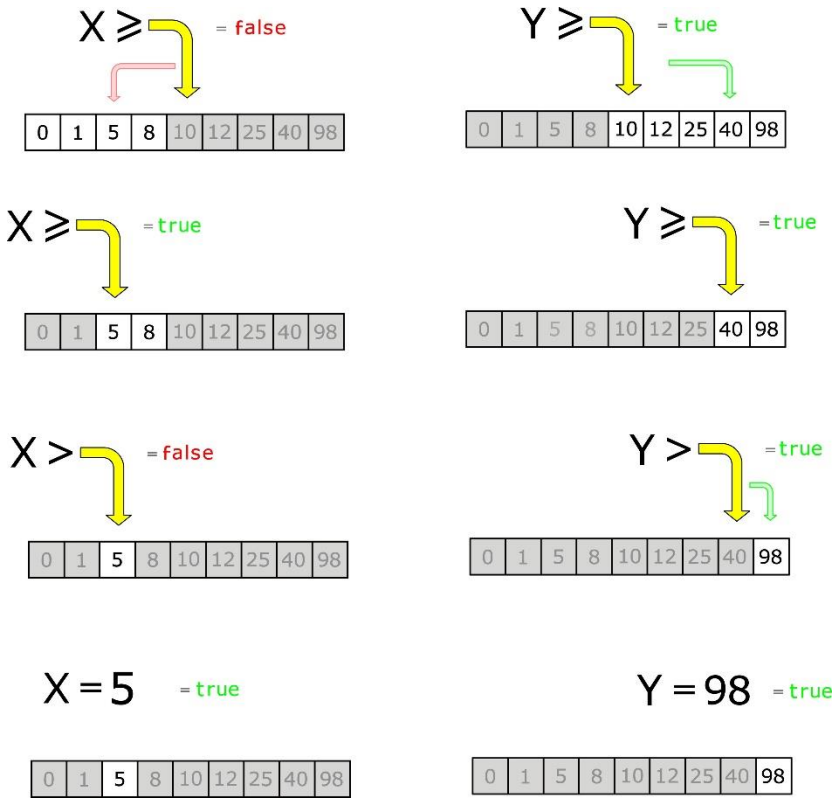
- No requiere que los datos estén ordenados para funcionar.
- Funciona sobre cualquier estructura de datos (arreglos, listas, archivos de flujo).

- **Desventajas:**
  - Muy lenta para bases de datos masivas, con una complejidad de .
  - El tiempo de búsqueda aumenta proporcionalmente con cada nuevo dato añadido.

  5. Binary Search (Búsqueda Binaria)

  Divide repetidamente a la mitad el intervalo de búsqueda en una lista ya organizada.

- **Ventajas:**
  - Increíblemente rápida, con una complejidad de .
  - Para 1,000,000 de elementos, solo necesita realizar un máximo de 20 comparaciones.
- **Desventajas:**
  - **Obligatorio:** Los datos deben estar ordenados previamente.
  - Requiere acceso aleatorio (arrays), por lo que es ineficiente en listas enlazadas.



Shutters

6. Jump Search

(Búsqueda por Saltos)

Busca en bloques de tamaño fijo para encontrar el intervalo donde se encuentra el elemento.

- **Ventajas:**
  - Mejor que la búsqueda lineal, con una complejidad de .
  - Útil en sistemas donde "saltar hacia atrás" es una operación costosa o no permitida.
- **Desventajas:**
  - Más lenta que la búsqueda binaria.
  - También requiere que el arreglo esté ordenado.

8. Conclusión General del Análisis

Tras evaluar el rendimiento de diversos algoritmos de ordenamiento y búsqueda en entornos de **C** y **Python**, los resultados confirman que la eficiencia de un programa no depende únicamente del hardware, sino fundamentalmente del **diseño algorítmico** y la **gestión de recursos** del lenguaje.

1. La Supremacía de la Complejidad Asintótica

El análisis demuestra que para volúmenes de datos masivos (n >= 10^5), la complejidad asintótica es el factor determinante. Mientras que **Bubble Sort** (O(n^2)) se vuelve impracticable debido a su crecimiento exponencial de tiempo, algoritmos como **Quick Sort** y **Merge Sort** (O(n log n)) mantienen una escalabilidad lineal-logarítmica que permite procesar millones de registros en segundos. En el ámbito de búsqueda, la **Búsqueda Binaria** (O(log n)) redefine la eficiencia, logrando encontrar elementos en microsegundos independientemente del tamaño del arreglo, siempre que se cumpla la condición de orden previo.

9. 2. Contraste entre C y Python

La comparativa de lenguajes revela un compromiso (*trade-off*) entre facilidad de desarrollo y rendimiento:

C (C89): Exhibe una superioridad técnica en el procesamiento masivo. Al ser un lenguaje compilado con acceso directo a la memoria y sin el "overhead" de objetos complejos, su ejecución es entre **10 y 100 veces más rápida** que la de Python. El consumo de memoria es predecible y eficiente (4 bytes por entero), lo que lo posiciona como la opción ideal para sistemas embebidos y computación de alto rendimiento.

Python: Aunque ofrece una sintaxis elegante y un desarrollo más rápido, su naturaleza interpretada y la gestión de memoria basada en objetos (donde un entero puede ocupar hasta 28 bytes) limitan su capacidad para manejar arreglos de 10^7 elementos bajo restricciones estrictas de RAM y tiempo de CPU.

3. Recomendaciones de Ingeniería

Para el desarrollo de software escalable, se concluye lo siguiente:

- **Priorizar el Orden:** La inversión de tiempo en un ordenamiento de $O(n \log n)$ se recupera con creces gracias a la velocidad de la búsqueda binaria posterior.
- **Elegir el Algoritmo Según los Datos:** Para arreglos pequeños o casi ordenados, la simplicidad de la **Inserción** o **Burbuja** puede ser aceptable, pero para "Big Data", la recursión eficiente es obligatoria.
- **Selección del Lenguaje:** Si el tiempo de respuesta y el consumo de memoria son críticos para el negocio, la implementación en **C** es la decisión técnica correcta.

"En el mundo del desarrollo, el código más rápido no es el que corre en el procesador más potente, sino el que sabe evitar las operaciones innecesarias."

Bibliografía

- Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Sedgewick, R. (1998). Algorithms in C, Parts 1-4: Fundamentals, Data Structures, Sorting, Searching (3rd ed.). Addison-Wesley.
- Knuth, D. E. (1997). The Art of Computer Programming, Volume 3: Sorting and Searching (2nd ed.). Addison-Wesley.
- Brassard, G., & Bratley, P. (1996). Fundamentals of Algorithmics. Prentice Hall.
- Skiena, S. S. (2008). The Algorithm Design Manual. Springer.